

Hit-Quota Load Balancer with Nginx + Ngrok

Phase 1 accepts 10 hits, Phase 2 accepts 5 hits, then Phase 3
Complete step-by-step guideline with explanations



Covers: Ubuntu/Debian Linux (native) and Docker Compose (Windows / macOS / Linux)
Follow-up to the first guide (round-robin Nginx + Ngrok). Estimated time: 30-40 minutes

Contents

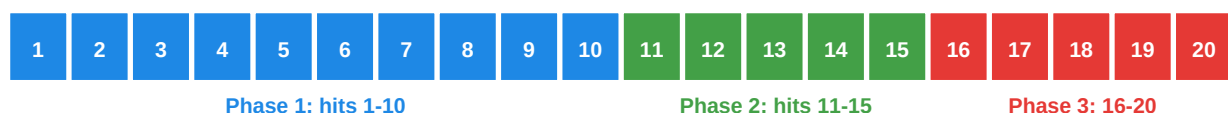
Section	What you will do
1. Goal and rules	Exactly how the hits move between the phases
2. How the solution works	Why a counter is needed and how the pieces fit together
3. Prerequisites	What you need before starting
Step 1	Install Nginx, Python and check Nginx modules
Step 2	Create the three web servers (Phase 1, 2, 3)
Step 3	Create the hit-counter service (the brain)
Step 4	Configure Nginx to route by the counter
Step 5	Test the 10 / 5 / 5 rule locally
Step 6	Install Ngrok and publish the load balancer
Step 7	Share the link, verify and reset the demo
8. Customizing the rules	Change quotas, make Phase 3 final, and more
9. Troubleshooting	Fixes for common problems
10. Cleanup and security	Shut everything down safely
Appendix A	Docker Compose version

1. Goal and rules

This project keeps the same base as the first guide: Nginx in front of three web servers, published to the internet with Ngrok. The change is **how** requests are distributed. Instead of rotating on every refresh, each phase now keeps receiving hits until its **quota** is used up:

Phase	Quota	Rule
Phase 1 (blue)	10 hits	Hits 1 to 10 are served by Phase 1. Hit 11 moves to Phase 2.
Phase 2 (green)	5 hits	Hits 11 to 15 are served by Phase 2. Phase 2's 6th hit (overall hit 16) moves to Phase 3.
Phase 3 (red)	5 hits (assumed)	Hits 16 to 20 are served by Phase 3. Overall hit 21 returns to Phase 1 and the cycle repeats.

Hit 11 switches Phase 1 -> Phase 2. Hit 16 (Phase 2's 6th) switches Phase 2 -> Phase 3. Hit 21 starts again at Phase 1.



One full cycle = 20 hits

Assumption you can change

Your request did not say how many hits Phase 3 accepts or what happens after it. This guide assumes **5 hits for Phase 3, then the cycle restarts at Phase 1**. Both are controlled by one line of the counter script (QUOTAS = [10, 5, 5]); Section 8 shows how to change the Phase 3 quota or make Phase 3 the permanent final phase.

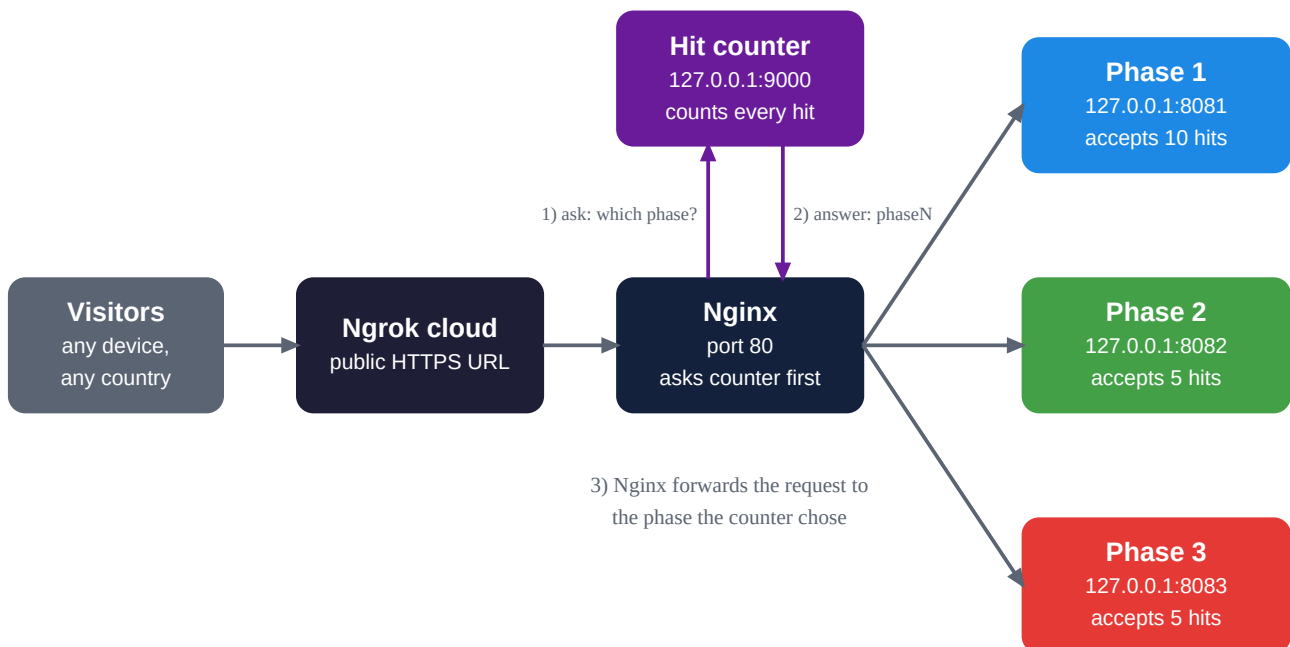
What counts as a hit?

One page load (a normal GET request to the address) is one hit, from any visitor anywhere. The counter is **global**, so if 3 people refresh at the same time they together consume the same quota. The browser's automatic favicon request is excluded.

2. How the solution works

Why plain Nginx cannot do this. Nginx's built-in balancing methods (round-robin, weights, least_conn, ip_hash) decide per request using rotation or connection load. Even with `weight=10` and `weight=5` Nginx mixes servers smoothly (for example 1,1,2,1,1,2...) instead of sending 10 hits to Phase 1 and only then moving on. A strict "first 10, then next 5" rule needs a **memory** of how many hits have happened.

The design. A tiny counter program remembers the hit number. For every visitor request, Nginx first asks the counter which phase should answer, then forwards the request to that phase. Nginx does this with its built-in `auth_request` feature, so no third-party Nginx modules are needed.



The components

Component	Port	Role
Phase 1, 2, 3 web servers	8081, 8082, 8083	Three Nginx server blocks, each serving one HTML page. Listen on 127.0.0.1 only.
Hit counter (Python)	9000	Keeps a running hit number. For each hit it answers with the chosen phase, the hit number, the hit inside the phase and the phase quota.
Nginx load balancer	80	Asks the counter, picks the matching phase, forwards the request, and writes the hit numbers into the page.
Ngrok	-	Publishes port 80 as a public HTTPS address.

Request flow for one hit

- A visitor opens the public Ngrok link. Ngrok forwards the request to Nginx on port 80.
- Nginx pauses the request and sends a quick internal call to the counter (/hit).
- The counter adds 1 to the total and calculates the position in the 20-hit cycle, then replies with headers such as X-Phase: phase2.
- Nginx reads that header and proxies the visitor's request to the chosen phase server.
- The phase's page is returned. Nginx replaces placeholders in the HTML so the page shows lines like *Phase hit 3 of 5 | Total hit 13*.

3. Prerequisites

Requirement	Details
Operating system	Ubuntu 20.04+ or Debian 11+ (native steps). Windows users: WSL2 (Ubuntu) or Appendix A (Docker).
Privileges	A user with sudo rights
Software	Nginx, Python 3 (standard library only; nothing to pip install), curl
Ngrok account	Free account at ngrok.com and your authtoken from the dashboard
Free ports	80, 8081, 8082, 8083 and 9000 must be unused
Internet	Outbound access only; no router or firewall changes needed for Ngrok

STEP 1

Install Nginx, Python and check modules

Install the packages and start Nginx.

```
sudo apt update
sudo apt install -y nginx python3 curl
sudo systemctl enable --now nginx
```

This solution needs two standard Nginx modules: `http_auth_request_module` (to ask the counter) and `http_sub_module` (to show the hit numbers on the page). Verify that both are present:

```
nginx -V 2>&1 | grep -o -E "http_auth_request_module|http_sub_module"
python3 --version
```

You should see both module names listed and Python 3.x. Ubuntu and Debian packages include them, as does the official `nginx:alpine` Docker image. If one is missing, install the package `nginx-full` (Debian/Ubuntu) and check again.

STEP 2

Create the three web servers

2.1 Create the Phase pages

This loop creates a folder and a colour-coded page for each phase. Each page contains three placeholders (`__PHIT__`, `__PQ__`, `__HIT__`) that Nginx will replace with live numbers in Step 4. The dollar-variables (`$i`, `$C`) are filled in by the shell so you only write the template once.

```
sudo mkdir -p /var/www/phase1 /var/www/phase2 /var/www/phase3
COLORS=( "#1e88e5" "#43a047" "#e53935" )

for i in 1 2 3; do
```

```

C=${COLORS[$((i-1))]}
sudo tee /var/www/phase$i/index.html > /dev/null <<EOF
<!DOCTYPE html>
<html lang="en"><head><meta charset="utf-8">
<title>Phase $i</title>
<style>
body{margin:0;height:100vh;display:flex;align-items:center;
justify-content:center;font-family:Arial,sans-serif;
background:$C;color:#fff;text-align:center}
h1{font-size:7rem;margin:0}
p{font-size:1.4rem;margin:.4rem 0}
</style></head>
<body><div><h1>PHASE $i</h1>
<p>Served by web server $i (port 808$i)</p>
<p><b>Phase hit __PHIT__ of __PQ__ | Total hit __HIT__</b></p>
</div></body></html>
EOF
done

```

2.2 Define the three backend servers

Create `/etc/nginx/conf.d/phase-backends.conf` (use `sudo nano`). Each block is a tiny web server that serves one folder on one port. Binding to `127.0.0.1` means the phases cannot be opened directly from outside; all traffic must pass through the load balancer, so every hit is counted.

```

# /etc/nginx/conf.d/phase-backends.conf
server {
    listen 127.0.0.1:8081;
    server_name _;
    root /var/www/phase1;
    index index.html;
    add_header X-Backend "phase1" always;
}
server {
    listen 127.0.0.1:8082;
    server_name _;
    root /var/www/phase2;
    index index.html;
    add_header X-Backend "phase2" always;
}
server {
    listen 127.0.0.1:8083;
    server_name _;
    root /var/www/phase3;
    index index.html;
    add_header X-Backend "phase3" always;
}

```

If you open a backend directly (for example `http://127.0.0.1:8081`) you will see the placeholder text such as `__HIT__` because only the load balancer fills it in. That is expected.

STEP 3

Create the hit-counter service

The counter is a small Python program using only the standard library. It answers three URLs, all reachable only from the machine itself:

URL	Purpose
<code>/hit</code>	Counts one hit and returns the chosen phase in headers. Called automatically by Nginx.
<code>/status</code>	Shows how many hits so far and which phase comes next. Does not count a hit.
<code>/reset</code>	Sets the counter back to zero so the demo starts again at Phase 1, hit 1.

3.1 Create the script

The rule lives in QUOTAS. The function `locate()` takes the number of hits so far, finds the position inside the 20-hit cycle (hit count modulo 20) and walks through the quotas to find the phase. A lock makes sure two simultaneous visitors never receive the same hit number.

```

sudo mkdir -p /opt/phase-counter
sudo tee /opt/phase-counter/counter.py > /dev/null <<'EOF'
#!/usr/bin/env python3
import json, os, threading
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

QUOTAS = [10, 5, 5]          # hits accepted by Phase 1, Phase 2, Phase 3
HOST = os.environ.get("BIND", "127.0.0.1")
PORT = 9000
lock = threading.Lock()
total = 0                    # hits handed out so far

def locate(n):
    # n = hits that happened BEFORE this one
    pos = n % sum(QUOTAS)    # position inside the current cycle
    for i, quota in enumerate(QUOTAS):
        if pos < quota:
            return i + 1, pos + 1, quota    # phase, hit in phase, quota
        pos -= quota

class Handler(BaseHTTPRequestHandler):
    def reply(self, code=200, body=b"", headers=None):
        self.send_response(code)
        self.send_response(code)
        for k, v in (headers or {}).items():
            self.send_header(k, v)
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        if self.command != "HEAD":
            self.wfile.write(body)

    def do_GET(self):
        global total
        if self.path.startswith("/hit"):
            with lock:
                phase, ph, quota = locate(total)
                total += 1
                n = total
                self.reply(200, b"", {"X-Phase": "phase%d" % phase,
                                     "X-Hit": str(n), "X-Phase-Hit": str(ph),
                                     "X-Phase-Quota": str(quota)})
        elif self.path.startswith("/reset"):
            with lock:
                total = 0
                self.reply(200, b"counter reset\n")
        elif self.path.startswith("/status"):
            with lock:
                n = total
                phase, ph, quota = locate(n)
                info = {"hits_so_far": n, "next_hit": n + 1,
                       "next_phase": phase, "next_phase_hit": ph,
                       "quotas": QUOTAS}
                self.reply(200, (json.dumps(info) + "\n").encode())
        else:
            self.reply(404)

    do_HEAD = do_GET
    do_POST = do_GET
    def log_message(self, *args):
        pass

ThreadingHTTPServer((HOST, PORT), Handler).serve_forever()
EOF

```

3.2 Run it as a background service

A systemd service starts the counter automatically and restarts it if it crashes. Create the unit file, then enable it:

```
sudo tee /etc/systemd/system/phase-counter.service > /dev/null <<'EOF'
[Unit]
Description=Phase hit counter
After=network.target

[Service]
ExecStart=/usr/bin/python3 /opt/phase-counter/counter.py
Restart=always
User=nobody

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now phase-counter
```

3.3 Test the counter by itself

```
curl -s http://127.0.0.1:9000/status
curl -si http://127.0.0.1:9000/hit | grep -i "^X-"
curl -s http://127.0.0.1:9000/reset
```

The first command prints a JSON line with `hits_so_far`. The second counts one hit and shows X-Phase : phase1 with X-Hit : 1. The third resets the counter to zero. Always reset after testing so the real demo begins cleanly.

Note

The hit count is kept in memory. Restarting the service or rebooting the machine sets the count back to zero.

STEP 4

Configure Nginx to route by the counter

4.1 Remove the default site

The default site also listens on port 80 and would conflict with the load balancer.

```
sudo rm -f /etc/nginx/sites-enabled/default
```

4.2 Create the load balancer configuration

Create `/etc/nginx/conf.d/load-balancer.conf`:

```
# /etc/nginx/conf.d/load-balancer.conf
upstream phase1 { server 127.0.0.1:8081; }
upstream phase2 { server 127.0.0.1:8082; }
upstream phase3 { server 127.0.0.1:8083; }

server {
    listen 80 default_server;
    server_name _;

    # Browsers request the favicon automatically; do not count it as a hit
    location = /favicon.ico {
        access_log off;
        return 204;
    }

    # Internal helper: ask the counter which phase should answer
    location = /_counter {
        internal;
```

```

    proxy_pass http://127.0.0.1:9000/hit;
    proxy_pass_request_body off;
    proxy_set_header Content-Length "";
}

location / {
    auth_request /_counter;
    auth_request_set $phase $upstream_http_x_phase;
    auth_request_set $hit $upstream_http_x_hit;
    auth_request_set $phit $upstream_http_x_phase_hit;
    auth_request_set $pquota $upstream_http_x_phase_quota;

    proxy_pass http://$phase;
    proxy_http_version 1.1;
    proxy_set_header Connection "";
    proxy_set_header Accept-Encoding "";
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;

    # Write the live numbers into the page
    sub_filter '__HIT__' '$hit';
    sub_filter '__PHIT__' '$phit';
    sub_filter '__PQ__' '$pquota';
    sub_filter_once off;

    add_header Cache-Control "no-store, no-cache, must-revalidate" always;
    add_header X-Hit $hit always;
    add_header X-Phase-Hit $phit always;
}
}

```

4.3 What each part does

Directive	Explanation
upstream phase1/2/3	Names each backend. Each group holds one server so traffic goes exactly where the counter says.
location = /favicon.ico	Answers the browser's icon request with an empty reply (204) so it is not counted as a hit and does not use up quota.
location = /_counter	An <i>internal</i> address (not reachable by visitors) that forwards to the counter's /hit endpoint without the request body.
auth_request /_counter	Makes Nginx call /_counter before serving the page. A 200 reply means "go ahead"; any error stops the request.
auth_request_set	Copies the counter's reply headers into Nginx variables (\$phase, \$hit, \$phit, \$pquota).
proxy_pass http://\$phase	Forwards the request to the upstream group whose name is in \$phase, such as phase2.
Accept-Encoding ""	Asks the backend for uncompressed HTML so the text replacement can work.
sub_filter	Replaces the placeholders in the HTML with the real hit numbers. sub_filter_once off replaces every occurrence.
Cache-Control no-store	Stops browsers from re-showing an old page, so each refresh reaches Nginx and counts as a hit.

4.4 Validate and reload

```
sudo nginx -t
sudo systemctl reload nginx
```

The test must print `syntax is ok` and `test is successful`. If it reports an error, note the file and line number and fix that line.

STEP 5**Test the 10 / 5 / 5 rule locally**

First reset the counter, then send 20 requests and count how many each phase served. The `uniq -c` part groups consecutive identical lines and prints how many there were.

```
curl -s http://127.0.0.1:9000/reset
for i in $(seq 1 20); do curl -s http://localhost | grep -o "PHASE [0-9]"; done | uniq -c
```

Expected result:

```
10 PHASE 1
5 PHASE 2
5 PHASE 3
```

Now send one more request. It must return to Phase 1 because overall hit 21 starts a new cycle:

```
curl -s http://localhost | grep -o "PHASE [0-9]" # prints PHASE 1
```

To watch the boundaries (hits 10, 11, 15, 16) print the numbers shown on the page. Reset first, then run:

```
curl -s http://127.0.0.1:9000/reset
for i in $(seq 1 17); do
  curl -s http://localhost | grep -o -E "PHASE [0-9]|Phase hit [0-9]+ of [0-9]+" | paste -sd ' '
done
```

Key lines of the output:

Overall hit	Expected output	Meaning
10	PHASE 1 Phase hit 10 of 10	Last hit Phase 1 accepts
11	PHASE 2 Phase hit 1 of 5	Phase 1 changes to Phase 2
15	PHASE 2 Phase hit 5 of 5	Last hit Phase 2 accepts
16	PHASE 3 Phase hit 1 of 5	Phase 2 (6th hit) changes to Phase 3
17	PHASE 3 Phase hit 2 of 5	Phase 3 continues

Finally open `http://localhost` in a browser and refresh. You should see Phase 1 (blue) for ten refreshes, then Phase 2 (green) for five, then Phase 3 (red). **Reset the counter again** with `curl -s http://127.0.0.1:9000/reset` before the real demo.

Counting tips

Use plain GET requests when testing. `curl -I` (HEAD) and browser pre-loading also count as hits. To inspect the counter without counting, use `/status`. Do not continue until this step works.

STEP 6**Install Ngrok and publish the load balancer****6.1 Create an account and get your authtoken**

- Sign up at ngrok.com and log in to the dashboard.
- Open the **Getting Started / Setup** page and copy your authtoken. Treat it like a password.

6.2 Install the Ngrok agent

Use the install command shown on the dashboard's setup page, because repositories and download links change over time. Common options:

System	Install method
Ubuntu / Debian	Follow the apt repository commands on the dashboard setup page, or download the Linux archive from ngrok.com/download and move the ngrok binary to <code>/usr/local/bin</code>
macOS	<code>brew install ngrok</code>
Windows	Download from ngrok.com/download or use the Microsoft Store / winget / Chocolatey

```
ngrok version
```

6.3 Add your authtoken (one time)

```
ngrok config add-authtoken YOUR_AUTHTOKEN_HERE
```

6.4 Start the tunnel

Tunnel the **load balancer port (80)**, never a backend port and never the counter port; otherwise hits would bypass the counter.

```
ngrok http 80
```

Ngrok shows a status screen like this:

```
Session Status   online
Web Interface    http://127.0.0.1:4040
Forwarding       https://a1b2-c3d4.ngrok-free.app -> http://localhost:80
```

The **Forwarding** address ending in `.ngrok-free.app` (or another Ngrok domain) is your public link. Keep this terminal open; closing it stops the tunnel.

Optional: a stable address

Free random URLs change every restart. In the dashboard under Domains you can claim a static domain and start with `ngrok http --url=YOUR-DOMAIN.ngrok-free.app 80`. Availability depends on your plan.

STEP 7

Share the link, verify and reset the demo

- **Reset first:** `curl -s http://127.0.0.1:9000/reset` so the next visitor receives Phase 1, hit 1. (Do this after any local tests.)
- Copy the https Forwarding URL and open it on a device that is **not** on your Wi-Fi (for example a phone on mobile data).
- Free Ngrok accounts show a one-time **warning page** to browser visitors. Clicking **Visit Site** continues. That interstitial is served by Ngrok and is not counted.
- Refresh repeatedly: hits 1-10 show Phase 1, hits 11-15 Phase 2, hits 16-20 Phase 3, hit 21 Phase 1 again.
- Watch progress any time with `curl -s http://127.0.0.1:9000/status`, and watch each request live at `http://127.0.0.1:4040`.

You can also test the public link from a terminal (the header skips the browser warning page):

```
for i in $(seq 1 11); do
  curl -s -H "ngrok-skip-browser-warning: 1" https://YOUR-URL.ngrok-free.app \
    | grep -o "PHASE [0-9]"
done
```

Shared counter

Every visitor, including you and any crawler that finds the link, uses the same global counter. With several people online, each one sees only some of the hits, so a person may jump from Phase 1 to Phase 2 after fewer than 10 of their own refreshes. Test alone for a clean demonstration.

8. Customizing the rules

Edit the script with `sudo nano /opt/phase-counter/counter.py`, then apply the change with `sudo systemctl restart phase-counter` (this also resets the counter to zero). No Nginx change is needed.

Goal	Change
Different quotas (for example 20, 10, 5)	Edit the line <code>QUOTAS = [20, 10, 5]</code> . The pages automatically show the new quotas.
Phase 3 accepts more or fewer hits	Change the third number in <code>QUOTAS</code> .
Phase 3 is final and keeps all later hits	Replace <code>pos = n % sum(QUOTAS)</code> with <code>pos = min(n, sum(QUOTAS) - 1)</code> . After the quotas are used, every hit goes to Phase 3 (page shows the last phase-hit number) until you reset.
Add a fourth phase	Add a fourth folder, backend (port 8084), upstream phase4, one more number in <code>QUOTAS</code> , and the matching colour page.
Start a fresh run without restarting	Call <code>curl -s http://127.0.0.1:9000/reset</code>

9. Troubleshooting

Symptom	Likely cause and fix
500 Internal Server Error on the main page	The counter is not running or unreachable. Check <code>sudo systemctl status phase-counter</code> and <code>curl -s http://127.0.0.1:9000/status</code> . Restart with <code>sudo systemctl restart phase-counter</code> .
Page shows __HIT__ or __PQ__ text	You opened a backend port directly, or the <code>sub_filter</code> directives are missing or <code>http_sub_module</code> is absent. Use the load balancer address and re-check Step 1 and Step 4.
nginx -t: unknown directive auth_request	Nginx was built without <code>http_auth_request_module</code> . Install <code>nginx-full</code> or use the Docker appendix.
Welcome to nginx page	Default site still enabled. Run <code>sudo rm -f /etc/nginx/sites-enabled/default</code> and reload.
502 Bad Gateway	A phase backend is not listening. Check <code>sudo ss -tlnp grep -E "808 9000"</code> ; ports 8081, 8082, 8083 and 9000 must appear.
403 Forbidden on a phase	Fix permissions: <code>sudo chmod -R 755 /var/www/phase*</code> and make sure <code>index.html</code> exists in each folder.
Switch happens too early (before 10 hits)	Extra requests are being counted (other visitors, HEAD requests, browser prefetch, earlier tests). Reset, test with one client, and check <code>/status</code> between steps.
Same page after refresh	Cached page. Confirm the Cache-Control header exists (<code>curl -sI http://localhost</code>) and try Ctrl+Shift+R.
Count restarted unexpectedly	The counter service was restarted or the machine rebooted; the count is in memory only.
Address already in use	Another program uses port 80 or 9000. Stop it or change the port, and tunnel the matching port with Ngrok.
Ngrok authentication failed	Authtoken missing or wrong. Re-run the <code>add-authtoken</code> command with the token from the dashboard.
Ngrok URL loads a warning page	Expected on free accounts. Click Visit Site, or send the <code>ngrok-skip-browser-warning</code> header from scripts.
Ngrok URL offline or changed	The tunnel stopped or restarted with a new URL. Check the terminal for the current Forwarding address.

Useful log commands:

```
sudo tail -f /var/log/nginx/access.log
sudo tail -f /var/log/nginx/error.log
sudo journalctl -u phase-counter -f
```

10. Cleanup and security

- **Stop the tunnel** with Ctrl+C in the Ngrok terminal when you finish; the public link stops at once.
- The counter listens on 127.0.0.1 only and Nginx marks `/_counter` as internal, so visitors cannot reset or read the counter through the public link.
- Never share your Ngrok authtoken in screenshots, chats or public repositories. Regenerate it in the dashboard if it leaks.
- Anyone with the link can open the pages and consume hits. Share it only when you want the demo to run.

To remove everything created in this guide:

```
sudo systemctl disable --now phase-counter
sudo rm /etc/systemd/system/phase-counter.service
sudo rm -rf /opt/phase-counter /var/www/phase1 /var/www/phase2 /var/www/phase3
sudo rm /etc/nginx/conf.d/phase-backends.conf /etc/nginx/conf.d/load-balancer.conf
sudo systemctl daemon-reload
sudo nginx -t && sudo systemctl reload nginx
```

Appendix A: Docker Compose version

Use this on Windows or macOS (Docker Desktop) or any Linux host with Docker. It runs five containers: three phase servers, the counter and the Nginx load balancer. Ngrok still runs on the host and tunnels port 8080.

A.1 Folder, pages and counter

```
mkdir phase-lb && cd phase-lb
mkdir phase1 phase2 phase3
COLORS=("#1e88e5" "#43a047" "#e53935")
for i in 1 2 3; do
  C=${COLORS[$((i-1))]}
  cat > phase$i/index.html <<EOF
<!DOCTYPE html>
<html lang="en"><head><meta charset="utf-8"><title>Phase $i</title>
<style>body{margin:0;height:100vh;display:flex;align-items:center;
justify-content:center;font-family:Arial,sans-serif;background:$C;
color:#fff;text-align:center}h1{font-size:7rem;margin:0}
p{font-size:1.4rem;margin:.4rem 0}</style></head>
<body><div><h1>PHASE $i</h1><p>Served by web server $i</p>
<p><b>Phase hit __PHIT__ of __PQ__ | Total hit __HIT__</b></p>
</div></body></html>
EOF
done
```

Save the Python program from Step 3.1 (everything between the two EOF lines) as `counter.py` in this folder. On Windows, create the folders and files by hand with the same content.

A.2 nginx.conf

```

upstream phase1 { server phase1:80; }
upstream phase2 { server phase2:80; }
upstream phase3 { server phase3:80; }

server {
    listen 80;
    location = /favicon.ico { access_log off; return 204; }

    location = /_counter {
        internal;
        proxy_pass http://counter:9000/hit;
        proxy_pass_request_body off;
        proxy_set_header Content-Length "";
    }

    location / {
        auth_request /_counter;
        auth_request_set $phase $upstream_http_x_phase;
        auth_request_set $hit $upstream_http_x_hit;
        auth_request_set $phit $upstream_http_x_phase_hit;
        auth_request_set $pquota $upstream_http_x_phase_quota;

        proxy_pass http://$phase;
        proxy_set_header Host $host;
        proxy_set_header Accept-Encoding "";
        sub_filter '__HIT__' '$hit';
        sub_filter '__PHIT__' '$phit';
        sub_filter '__PQ__' '$pquota';
        sub_filter_once off;
        add_header Cache-Control "no-store, no-cache, must-revalidate" always;
    }
}

```

A.3 docker-compose.yml

```

services:
  counter:
    image: python:3.12-alpine
    command: python /app/counter.py
    environment:
      BIND: "0.0.0.0"
    volumes: ["/counter.py:/app/counter.py:ro"]
  phase1:
    image: nginx:alpine
    volumes: ["/phase1:/usr/share/nginx/html:ro"]
  phase2:
    image: nginx:alpine
    volumes: ["/phase2:/usr/share/nginx/html:ro"]
  phase3:
    image: nginx:alpine
    volumes: ["/phase3:/usr/share/nginx/html:ro"]
  lb:
    image: nginx:alpine
    ports: ["8080:80"]
    volumes: ["/nginx.conf:/etc/nginx/conf.d/default.conf:ro"]
    depends_on: [counter, phase1, phase2, phase3]

```

A.4 Run, test and publish

```

docker compose up -d
for i in $(seq 1 20); do curl -s http://localhost:8080 | grep -o "PHASE [0-9]"; done | uniq -c
docker compose restart counter          # resets the hit count to zero
ngrok http 8080

```

The expected test result is the same as Step 5: 10 x Phase 1, 5 x Phase 2, 5 x Phase 3. Restarting the counter container resets the count before the real demo. To change quotas, edit QUOTAS in counter.py and run `docker compose restart counter`. Stop everything with `docker compose down`.